Why uploading is slow, and what to do about it

19 Oct 2026 · proposal, nothing changed yet

> *## Revised after measurement — read this first The boot log from production settles §8 and* ***disproves the headline finding.*** `getAllVideos (0 clips, blob URLs minted): +21ms` *— the local library on that browser is* ***empty****, so L1 costs* ***1 ms****, not seconds. First paint is 22–40 ms. The local read path is not the problem on that machine, and item 1 below is not where to start. L1 is still real as code — the arithmetic holds for a library that has clips in it — but it is dormant wherever the library is empty, and it should not be built on my reading alone.* ***Open question: is the browser that does the importing the same one these numbers came from?*** *If clips are imported on a machine whose library then holds thirty of them, L1 bites there and nowhere else. What the log does show, measured, is three different things — all on the* ***day-open*** *path, none of them on the upload path the complaint is about. They are in §9. It also shows what is* ***not*** *instrumented: the import and upload paths emit no timings at all, so nothing in §2's L3/L4/L5 or §3's N1–N5 is confirmed or denied by this data. See §10.*

The complaint is that uploading video and photos is slow, *including the local step* — USB drive to the browser, before a byte leaves the laptop. That split turns out to be the right way in: the local half and the network half are slow for entirely unrelated reasons, and the local half is both the worse offender and the cheaper fix.

Everything below is read off the code. Where I have estimated rather than measured, it says so. **Section 8 is the one measurement worth taking before any of this is built** — the app already logs what is needed.

---

1. The short version

	What	Where	Cost
L1	Every video blob is re-read from IndexedDB **3× at boot**, and once more per imported file	`localStore.js:276,519,544,556` · `SSAApp.jsx:531,535,543`	the dominant local cost; grows with library size, not with what you import
L2	`saveVideo` copies the whole file USB → IndexedDB	`localStore.js:267`	unavoidable *as designed*, but it need not be on the critical path
L3	All clips probe **concurrently and unbounded**, and the probe seeks to the end of the file	`UploadTab.jsx:169` · `videoProbe.js:254`	USB drives collapse under concurrent random reads
L4	Photo import is **strictly serial**, with two full-resolution main-thread decodes per photo	`photoStore.js:383`	~2 min each on the 170 MP compilations, by our own docs
L5	No `createImageBitmap`, no `OffscreenCanvas`, no Worker anywhere in the codebase	—	every decode blocks the UI thread
N1	The **background** originals queue still uses the old TUS→Stream path; storage-first exists but only the batch button uses it	`useCloudSync.js:291` vs `useBatchActions.js:260`	clips auto-uploaded are unwatchable for 60–120 min; clips uploaded by hand are instant
N2	Transcode and upload are serialised per clip when they could overlap	`useCloudSync.js:90`	~2× on the proxy queue, free
N3	Photo sync is serial across every date and photo — ~4 round trips each	`photoStore.js:527`	100 photos ≈ 400 sequential RTTs
N4	`photoStore.getCreds()` is **uncached** and called twice per photo, and I made that route more expensive last month	`photoStore.js:432`	see §7 — my regression
N5	`auth.getUser()` is called **inside** the originals loop	`useCloudSync.js:245`	~0.3–0.9 s × clip count

---

2. The local half — why USB → browser is slow

L1. The library is re-read in full, repeatedly

`idbGetAll(db, "videos")` returns every row **including its** `blob`. The blob is the clip. So every call deserialises the entire local video library out of IndexedDB and into memory.

It is called:

- **3× on every app boot** — `pruneInertVideos()`, then `dedupeVideos()`, then `getAllVideosForMembership()` (`SSAApp.jsx:531,535,543`);
- **once per file imported**, inside `saveVideo`, purely to find a possible duplicate by `clipKey` (`localStore.js:276`).

Import twenty clips into a library that already holds thirty and that is twenty full reads of thirty clips, on top of the three at boot. The work is proportional to **what is already stored**, not to what you are importing, which is exactly the shape of "it got slower over the season".

The duplicate check needs three fields — date, name, size. It reads hundred-megabyte blobs to get them.

Fix. Store `clipKey` as its own indexed field and look it up with `index.getKey()`, which returns a primary key and reads no record body. Then `get(id)` the one row that matched. For the boot path, `prune` and `dedupe` want metadata only: a cursor over an index, or a separate `videoMeta` store written alongside the blob, gives them that. This is the single highest-ratio change in the document — it is a lookup strategy, not an architecture.

> ***Caveat worth stating:*** *adding an index means bumping* `DB_VER`*, and CLAUDE.md is emphatic about what that costs — half a dozen files open* `ssa-db` *versionless and never listen for* `versionchange`*, so an upgrade can block for ever and hang every read behind it. The* `clipKey` *index can be avoided:* `getAllKeys()` *on the* `sessionDate` *index plus a* `get()` *per candidate is slower in theory and still reads no blobs for the 99 % case. I would take that route first and leave* `DB_VER` *alone.*

L2. The copy itself

`saveVideo` puts the `File` into IndexedDB (`localStore.js:317`). The browser reads every byte off the USB drive and writes it to the internal disk. For a day of drone footage that is tens of gigabytes of genuine I/O, and no code change removes it while local playback and local transcode are requirements.

What *can* change is when it happens. Today the import blocks on it. It could be queued behind first paint, so the library is browsable while the copy proceeds — the clip only needs its blob for playback, transcode or upload, all of which come later.

Worth asking separately: **does the original need to be in IndexedDB at all?** The File handle from the picker stays valid for the page's lifetime, and the File System Access API can persist a directory handle across sessions — which is already how "Watch clips" and "Review footage" work. A clip whose bytes stay on the SSD and are read on demand would make the import close to instant. That is a design change, not a tweak, and it trades away offline playback once the drive is unplugged.

L3. Unbounded probe fan-out, and a seek to the end of the file

`UploadTab.jsx:169` is `valid.forEach(async f => …)`. There is no pool: twenty files means twenty `probeVideo` calls in flight at once, each with its own `<video>` element and its own blob URL, plus twenty `extractVideoCreationTime` calls each reading 1 MB of head and 2 MB of tail.

Twenty concurrent readers on one USB drive is close to the worst possible access pattern — the drive seeks between twenty positions instead of streaming. A pool of 2–3 will very likely be *faster* than twenty, not just gentler.

Then `videoProbe.js:254`:

```
try { v.currentTime = 1e101 } catch { done() }
```

This is a deliberate, well-commented workaround: on a re-muxed MP4 `duration` is still `Infinity` at `loadedmetadata`, and seeking past the end forces the browser to resolve it. The duration is load-bearing — it is what distinguishes a start-of-recording stamp from an end-of-recording one. But the cost is that the browser may read a great deal of the file to answer, off the USB drive, twenty files at once.

The duration is already in the container. `_scanMvhd` (`videoProbe.js:6`) parses the `mvhd` atom, which carries `duration` and `timescale` — the file tells us directly. We read that atom anyway. Taking the duration from it would let the seek become a fallback for the rare file where the atom is missing, rather than the default path for every clip.

L4 / L5. Photo import: serial, and twice-decoded on the UI thread

`photoStore.js:383` is a plain `for … of` — one photo at a time, start to finish. Per photo:

1. `readExif` — cheap, headers only;
2. `convertToJpeg` — correctly short-circuits for JPEG (`photoStore.js:227`);
3. `idbPutPhoto` — the copy;
4. `generateLqip` — **decodes the full-resolution image** to produce a 20 px placeholder.

Then later, at upload time, `uploadThumb` calls `generateThumbnail`, which **decodes the full-resolution image again** for the 480 px thumbnail.

So each photo is decoded twice at full resolution, both times via `new Image()` and `drawImage` on the main thread. `docs/uploading-a-days-media.md` already records the consequence — "*≈2 min each — the thumbnail alone decodes 170 MP*" — and treats it as a fact of life. It is not.

Fix, in order of effect:

- **One decode, two outputs.** The 480 px thumbnail downsamples to the 20 px LQIP perfectly well. There is no reason to touch the original twice.
- `createImageBitmap(blob, { resizeWidth, resizeHeight, resizeQuality })`**.** This decodes *at the target size*, so a 170 MP JPEG never becomes a 170 MP bitmap in memory. It is also the only decode path that can run in a Worker. Neither `createImageBitmap` nor `OffscreenCanvas` nor `Worker` appears anywhere in the codebase today.
- **A Worker pool of 3–4.** Decode is pure CPU on independent inputs — the textbook case. Keeping it off the main thread also stops the import freezing the UI, which is half of what "slow" means to someone watching it.

Expect the large compilations to go from ~2 min to a few seconds each. This is the second-highest-ratio change here.

---

3. The network half

N1. Two original-upload paths, and the automatic one is the slow one

There are two implementations of "upload the full-resolution clip":

Path	Used by	Behaviour
`uploadOriginalStorageFirst`	`UploadTab.jsx:514`, `useBatchActions.js:260`	PUT to Bunny Storage, then Stream **fetches** it. Watchable immediately as progressive MP4
`uploadFileToStream` (TUS)	`useCloudSync.js:291`, `SSAApp.jsx:1704`	TUS straight to Stream. Unwatchable until Stream transcodes — **60–120 min on this library**

The storage-first path was built precisely to kill that wait. The **background auto-sync queue was never switched over.** So a clip that uploads by itself is unwatchable for an hour; the same clip uploaded via the batch button is watchable at once. Same bytes, same uplink — the difference is which function the queue happens to call.

This is not a performance tuning question, it is an unfinished migration, and it is the first thing I would change. It is also close to a drop-in: the signatures differ, but `useBatchActions.js:260` is a working call site to copy.

N2. Transcode and upload do not overlap

`useCloudSync.js:90` drains the proxy queue with `while (queue.length) { … await syncProxyForVideo(…) }`, and `syncProxyForVideo` transcodes *then* uploads. So the shape is:

```
transcode 1 ──▶ upload 1 ──▶ transcode 2 ──▶ upload 2 ──▶ …
```

Transcode is CPU, upload is network. They have no resource in common. The comment at `useCloudSync.js:50` justifies serialisation with "*ffmpeg.wasm is single-instance per page, parallel runs corrupt each other*" — which is true, and does not apply to the upload step at all, nor to the WebCodecs engine that is now the default (`video-proxy.ts:7`).

A one-deep pipeline — transcode N+1 while uploading N — keeps a single transcode in flight and still roughly halves wall-clock on a queue:

```
transcode 1 ──▶ transcode 2 ──▶ transcode 3
           └──▶ upload 1    └──▶ upload 2
```

N3 / N4 / N5. Per-item round trips

Photo sync (`photoStore.js:527`) is a nested serial loop over every date and every photo. Per photo it performs, in sequence:

1. `getCreds()` → `/api/storage/credentials`
2. PUT the thumbnail
3. `writeMeta` → PUT the `_meta.json`
4. `getCreds()` **again** → `/api/storage/credentials`
5. PUT the original
6. `mirror()` → Supabase upsert

Six serial round trips per photo, two of them avoidable outright. On a marina or hotel link at 80 ms RTT, 100 photos is ~10 minutes of pure latency before a single byte of payload is counted.

Fix. Cache the credentials (`bunny.js:16` and `bunny-storage-upload.ts:24` both already do — `photoStore` is the one that does not), and run the photos through a bounded pool of 4–6. Independent objects, no ordering requirement.

The same applies to `useCloudSync.js:245`, which calls `auth.getUser()` inside the originals loop. It is hoisted out everywhere else in the app for exactly this reason, with a comment saying so.

---

4. What Bunny offers that we are not using

Re-read today against the current docs; two things have changed since this pipeline was written.

Bunny Storage now has an S3-compatible API — with multipart

The HTTP API we use has no multipart, no resumability and one connection per object, which is why `bunny-storage-upload.ts:60` says a dropped connection means starting the object again. The S3 endpoint has `CreateMultipartUpload` / `UploadPart` / `CompleteMultipartUpload`, up to 10,000 parts, sessions valid for 10 days, and Bunny recommend it for objects over 100 MB. The published limits also say, in as many words: "*Up to 1 Gbps per connection. To go faster, split the transfer across multiple parallel connections.*"

That is the difference between one TCP stream to Falkenstein and eight. Over a high-latency link a single stream is window-limited well below the available uplink, which is the classic reason a 2 GB drone clip crawls on a connection that speed-tests fine.

It would also fix something I flagged last month and could not then solve: the S3 endpoint supports **presigned URLs** (1 second to 7 days). A server-side presigner would hand the browser per-part URLs and the zone write key would stop being shipped to the client at all — the thing `src/app/api/storage/credentials/route.ts` currently documents as unavoidable.

The catch, and it is a real one: S3 can only be enabled **when a storage zone is created**. It cannot be turned on for the existing zone — Bunny's docs say so explicitly and the CLI announcement repeats it. Taking this means creating a second zone and migrating, which given §2 of last month's work (every key now carries `teams/<team>/boats/<boat>/…` and reads fall back to the old layout) is more tractable than it would have been, but it is a project, not an afternoon. It is also still **Public Preview**.

I would not start here. I would do §5 first and come back to this once there is a measurement saying the uplink, rather than our own serialisation, is the limit.

Region

`BUNNY_STORAGE_REGION=de` — Falkenstein. Writes must go to the zone's primary region; replicas are read-only. Bunny has a `uk` region, and Baraka's October days were sailed in the UK. Whether that matters is a latency question I cannot answer from here, and it is not worth moving a zone over on a guess — but it is worth one `curl` timing test from the boat's actual connection.

TUS

`bunny.js:134` sets `retryDelays` but no `chunkSize`, so `tus-js-client` defaults to a single PATCH for the whole file. If the originals queue stays on TUS for any reason (I think it should not — see N1), a finite chunk size is what makes the retry logic useful rather than decorative. Bunny's TUS docs recommend `AuthorizationExpire` of at least an hour, which we do satisfy.

---

5. What I would do, in order

Ordered by effect per unit of risk. The first three are local, contained, and do not touch the network path at all.

#	Change	Expected effect	Risk
1	Stop `saveVideo` and the boot path reading blobs to answer metadata questions (L1)	Large and growing — removes work proportional to library size from both boot and every import	Low. No `DB_VER` bump needed if done with `getAllKeys` + `get`
2	One decode per photo, via `createImageBitmap` at target size, in a Worker pool (L4/L5)	170 MP compilations from ~2 min to seconds; UI stops freezing	Low–medium. New code path; needs a fallback for browsers without resize support
3	Bound the probe fan-out to 2–3, and take duration from `mvhd` instead of the seek (L3)	Likely faster *and* gentler on the drive	Low. The seek stays as the fallback
4	Point the background originals queue at `uploadOriginalStorageFirst` (N1)	Kills the 60–120 min unwatchable window for auto-uploaded clips	Low. Working call site to copy from
5	Cache credentials in `photoStore`; hoist `getUser()` out of the loop (N4/N5)	2 round trips per photo, 1 per clip	Trivial
6	Bounded pool of 4–6 for photo sync (N3)	Several-fold on a day of photos	Low
7	Overlap transcode and upload, one deep (N2)	~2× on the proxy queue	Medium — queue state machine
8	S3 multipart on a new zone, with presigned part URLs (§4)	The uplink ceiling itself; and gets the write key out of the browser	High — new zone, migration, still Public Preview

Items 1–6 are each self-contained and independently shippable. I would not bundle them.

---

6. What I am *not* proposing

Dropping the original upload. The original is ~80× the proxy's bytes (1.2 Mbps at 720p against 4K drone footage), so it dominates the network total and is the obvious thing to question. But it is the archive, and that is a product decision rather than a performance one. Worth a separate conversation about whether every clip needs full quality in the cloud or only the ones someone marks.

A service worker for background upload. It would let uploads continue with the tab closed, and it is a large piece of machinery to add to a pipeline whose current limits are all elsewhere.

---

7. A correction I owe

`photoStore.getCreds()` has never been cached, so photo sync has always made two calls to `/api/storage/credentials` per photo. Last month I added `requireActiveUser()` to that route — a Supabase auth check plus a `users` row query — to close it to anonymous callers.

That was the right fix for the security hole, and it made this path more expensive: those two calls per photo now each carry an auth round trip and a database query. For a 100-photo day that is 200 server calls that got measurably slower. I did not check the call sites' caching when I changed the route, and I should have.

It is cheap to put right (item 5 above), and the guard should stay.

---

8. Measure this first

Everything in §2 is read from code and arithmetic, not from a stopwatch. The app already logs what settles it — `SSAApp.jsx:483` writes `[boot] <label>: +Nms` for each boot phase, including `prune + dedupe` and `getAllVideos (N clips, blob URLs minted)`.

On the laptop, with the real library loaded, open the console and reload. Then send the `[boot]` lines. If the gap between `local sessions` and `getAllVideos` is seconds rather than milliseconds, L1 is confirmed as the dominant local cost and item 1 is unambiguously where to start.

Two more numbers worth having, both one-off:

- **Import one clip and time it** with the drive connected, then again with the same clip copied to the internal disk first. The difference separates "USB is slow" from "we are slow about USB".
- `curl -w` **a 10 MB PUT** to `de` and to `uk` from the boat's connection, to settle the region question in §4 rather than guessing.

I would rather start item 1 on evidence than on my reading of the code, even though I am fairly confident of the reading.

---

9. What the boot log actually measured (19 Oct, production)

Two page loads, second one warm. Local is fine; everything costly is cloud or CPU.

Phase	Load 1	Load 2	What it is
→ first paint	22 ms	40 ms	all local work, including L1
`checkCloudStatus`	1070 ms	361 ms	**a live Bunny folder listing**, in front of everything
`listSessionsCloud`	833 ms	322 ms	the session list
`listVideosCloud`	307 / 229 ms	269 / 408 ms	**two of them**

Phase	Load 1	Load 2	What it is
`getSessionCloud` (log + xml)	1445 / 1715 ms	982 / 1370 ms	**two of them**
`xml + timeline build`	~1.5 s	~1.5 s	pure CPU after the log lands
`tag list`	211 ms	190 ms	
day usable	**~5.3 s**	**~3.8 s**	

D1. `loadDate` runs twice, concurrently, on every load

Two of every `[loadDate]` line, each with its own elapsed clock — so two invocations, not double logging. **The day's log is downloaded twice.**

The cause is two independent paths both deciding to open the landing day:

- boot's `doCloud()` → `listSessionsCloud` → `loadDate` (`SSAApp.jsx:698`);
- `rescope()` in `useWorkspaceIdentity.js:151`, fired by `ssa:active-membership-changed`, which does its **own** `listSessionsCloud` and its own `loadDate`.

The comment at `useWorkspaceIdentity.js:116` already half-admits it: "*Cloud list will be refreshed by the boot effect's listSessionsCloud call when it re-runs; but to make the switch feel instant, also trigger one here.*" That is the right instinct for a **boat switch**, which is what the listener is for — it is wrong at boot, when the boot path is about to do the same work.

`loadDateSeqRef` ("latest wins") keeps the *state* correct, so this has never produced a visible bug. It does not stop either call paying for the network.

Fix. Have `rescope()` skip its landing load when boot has not finished, or give `loadDate` a short-lived in-flight map keyed by date so a second call for the same day joins the first instead of repeating it. The second is the more robust of the two and also covers the tab-switch effects at `SSAApp.jsx:264`.

Worth roughly **1.0–1.4 s and one full log download per day-open.**

D2. `checkCloudStatus` gates the cloud phase on a live Bunny round trip

`/api/cloud/status` is not the cheap config read its name suggests. Per call it does: `requireActiveUser()` (Supabase auth **+** a `users` row query — mine, added last month) and then **a root folder listing against Bunny Falkenstein** with a 4 s timeout (`route.ts:30`). Vercel → Supabase → Supabase → Germany → back, and `doCloud()` awaits it before it will list sessions.

361 ms warm, 1070 ms cold, every page load, for an answer that changes when the env vars change — i.e. never, within a session.

Fix. Nothing downstream needs it except a UI badge, so start it in parallel with `listSessionsCloud` rather than in front. Cache the verdict in `sessionStorage` for the tab. Both are small and independent.

D3. The timeline build is ~1.5 s of CPU

From `getSessionCloud` landing to `xml + timeline build` is ~1.5 s on both loads, with no network in between — so it is parsing and node-building on the main thread. It is the **largest single item** in the day-open budget and the one I know least about; it needs a profile before anyone guesses at it.

---

10. Revised order

Items 1–3 are new, measured, and all on the **day-open** path. Items 4+ are the upload-path items from §5, unchanged but now explicitly unmeasured.

#	Change	Evidence	Expected
1	De-duplicate `loadDate` (D1)	measured	~1.0–1.4 s + one log download per day-open
2	Un-gate / cache `checkCloudStatus` (D2)	measured	0.36–1.07 s per load
3	Profile the timeline build (D3)	measured	up to ~1.5 s, unknown until profiled
4	Point the background originals queue at storage-first (N1)	code-read	kills the 60–120 min unwatchable window
5	Cache credentials in `photoStore`, hoist `getUser()` (N4/N5)	code-read	2 RTT per photo, 1 per clip
6	One decode per photo via `createImageBitmap` in a Worker (L4/L5)	code-read + our own docs	170 MP compilations: ~2 min → seconds
7	Bound the probe fan-out; duration from `mvhd` (L3)	code-read	faster *and* gentler on the drive
8	Bounded pool for photo sync (N3)	code-read	several-fold on a day of photos
9	Blob-free metadata reads (L1)	**disproved on an empty library**	nothing until a library has clips
10	S3 multipart on a new zone (§4)	docs	the uplink ceiling itself

Items 4–8 are still the ones that address the stated complaint. They remain unmeasured, which is what §11 is for.

---

11. The measurement still missing

The boot log covers day-open. **It does not touch import or upload**, which is what the complaint is about, because neither path is instrumented.

Two ways to get it, in order of preference:

1. **Add the timers.** `SSAApp.jsx:483` and `loadDate`'s `_lm` are the pattern to copy; the places that want them are `handleVids` (per clip: probe, metadata scan, `saveVideo`), `importFiles` (per photo: exif, decode, IDB write, lqip) and the two queue drains in `useCloudSync` (per clip: transcode, upload, mark). That is a measurement-only change, no behaviour touched, and it would settle items 4–8 the way the boot log just settled 1–3.
2. **Or a stopwatch**, which needs no code: import one clip from the SSD and time it; copy the same clip to the internal disk and time it again. The difference separates "USB is slow" from "we are slow about USB" and is worth having either way.

I would rather instrument than guess twice.